

Verificador de Similaridade de Textos

Bruno de Paula Souza Ra:10439739

Gabriel Ferreira Ra:10442043

Gian Lucca Campanha Ra:10438361

1. Introdução

Comparar textos tem sido uma tarefa de extrema importância nos dias atuais. Seja para análises de plágio acadêmico, recomendações de conteúdo, organização de documentos e afins, todos dependem de algo que possa calcular a similaridade dos documentos de uma forma eficiente.

Esse projeto implementa um comparador de textos, retornando sua similaridade através de duas principais estruturas de dados: Hash Table, responsável pelo armazenamento dos vocabulários contidos nos documentos, e Árvore AVL, responsável por guardar os pares de documentos comparados pelo seu grau de similaridade.

O sistema processa os documentos, removendo Stop Words, espaços em branco e pontuações, e calcula a similaridades entre os pares utilizando a métrica Cosseno. Os resultados depois de armazenados na AVL podem ser consultados via lista a partir de um limiar informado, via Top K maiores similaridades ou via busca comparando dois documentos específicos.

Além de desenvolver um sistema funcional, o trabalho tem como objetivo analisar o desempenho das estruturas implementadas, comparando duas funções Hash diferentes e diferentes métricas de similaridade.

2. Metodologia

2.1 HashTable

Foram implementadas duas funções Hash: Somatória de ASCII e o Hash Polinomial.

- Somatória de ASCII: Itera por cada caractere da string e soma seus valores ASCII. No final, divide a soma pelo tamanho da tabela e pega o resto, garantindo que o índice sempre caiba dentro do array. Essa função pode trazer o mesmo índice para palavras com a mesma letra mas em ordens diferentes, causando maior número de colisões;

- Hash Polinomial: A cada caractere, multiplica o hash atual por um número primo (escolhido 31) e soma ao valor do caractere, depois divide o valor absoluto pelo tamanho da tabela, evitando índices negativos que surgem de overflow.

Decidimos seguir com o Hash Polinomial pela redução no número de colisões em arquivos com um vocabulário muito grande. Em arquivos com poucas palavras, o método de Somatória ASCII pode ter um desempenho melhor.

Também foi utilizado um endereçamento aberto para tratar as colisões, utilizamos a sondagem

linear que já estava implementada no código original do professor Joaquim. Optamos por seguir com a mesma pela simplicidade de implementação, e como o rehash é realizado após o fator de carga chegar em 0.75, conseguimos manter as colisões sobre controle.

2.2 Árvore AVL

Foi implementado um contador de rotações, onde conseguimos calcular quantas rotações simples ou duplas foram realizadas após todos os inserts das similaridades e pares. Esses contadores são incrementados pela função `balance` sempre que é realizado algum tipo de rotação, depois, o insert fica responsável por imprimir o valor desses contadores.

O `insertRecursive` fica responsável por percorrer a AVL comparando a similaridade com cada nó existente. Caso haja algum nó com a mesma similaridade que está sendo adicionada, o par de documentos é adicionado à lista do nó. Ao subir da recursão, atualiza o FB enquanto verifica a necessidade de rotações.

O `listarAcimaDoLimiar` é uma recursão que percorre a árvore de forma decrescente, indo do maior para o menor, a cada nó ele compara o valor da similaridade com a limiar definida pelo usuário e imprime somente os pares com similaridade iguais ou superiores à essa limiar.

O `listarAbaixoDoLimiar` faz exatamente o inverso, percorre a árvore de forma crescente e imprime tudo que está abaixo da limiar. Ambas funções são utilizadas dentro do modo lista de impressão.

O `topk` tem um funcionamento parecido com o `listarAcimaDoLimiar`, percorre a árvore de forma decrescente e imprime aquilo que está acima do limiar definido pelo usuário, porém, há um contador que é incrementado até que chegue no valor K que também é definido pelo usuário, quando chega a recursão para, fazendo com que seja imprimido somente o top K (top 2, top 3, etc).

2.3 Documento

Essa classe é responsável pelo tratamento do texto recebido. Ela remove as Stop Words, espaços em brancos e adiciona a palavra na Tabela Hash, caso a palavra já exista dentro da tabela, apenas incrementa a frequência dessa palavra.

2.4 Comparador de Documento

Aqui é onde definimos a métrica que será utilizada para definir a similaridade dos documentos, implementamos as 3 seguintes métricas:

- Cosseno: trata cada documento como um vetor, de palavras, onde cada dimensão é uma palavra e o valor é sua frequência. A similaridade é o cosseno do ângulo entre dois vetores, ou seja, quanto menor o ângulo, maior a similaridade entre eles;
- Jaccard: Trabalha apenas com o vocabulário do documento, dividindo a intersecção pela união dos vocabulários dos documentos;
- Dice: similar ao Jaccard, porém, divide o dobro da intersecção pela soma dos tamanhos dos documentos.

Decidimos utilizar o Cosseno pois ele leva em conta a frequência de cada palavra, trazendo um cálculo de similaridade mais preciso para textos.

2.5 Node

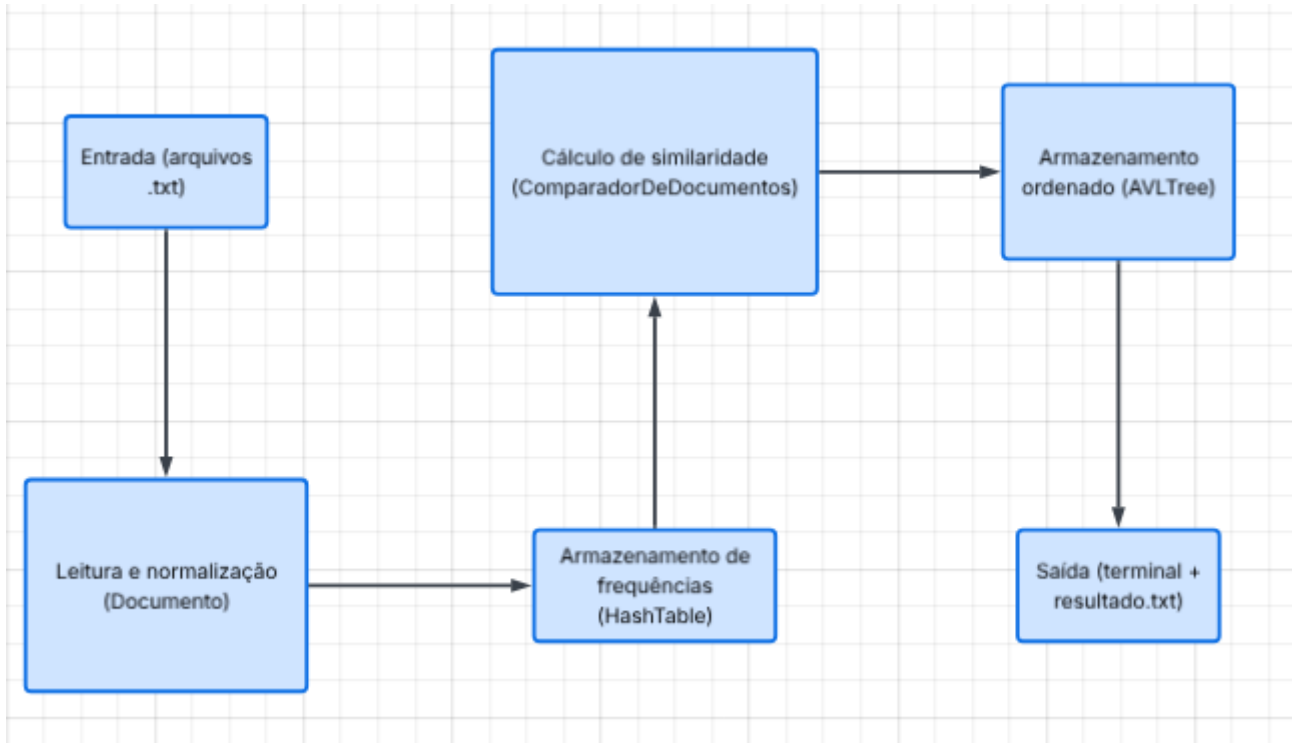
Foram realizadas pequenas alterações que diferem essa classe Node para uma Node comum:

Iniciamos uma lista de objetos Resultados para que fosse possível guardar pares dentro dos nós;

Alteramos o tipo do atributo data para que ele pudesse guardar o Double das similaridades.

3. Descrição do Algoritmo

3.1 Fluxo do Sistema



3.2 Pseudocódigos de funções mais complexas

3.2.1 Inserção na AVL

INSERIR(similaridade, par):

SE nó == nulo:

criar novo nó

adicionar par à lista do nó

retornar nó

SE similaridade < nó.valor:

descer para esquerda

SE similaridade > nó.valor:

descer para direita

SE similaridade == nó.valor:

adicionar par à lista do nó existente

atualizar fator de balanceamento

SE desbalanceado:

aplicar rotação adequada

retornar nó balanceado

3.2.2 Procurar posição da chave na HashTable

PROCURAR_POSICAO(chave):

 calcular posição inicial com hash(chave)

 i = 0

 ENQUANTO i < capacidade:

 calcular pos = (h + i) % capacidade

 SE i > 0 aumenta numero de colisões

 SE posição contém a chave buscada:

 retornar pos

 SE posição está vazia ou removida:

 retornar pos // candidata para inserção

 i++

lançar erro // tabela cheia

4. Análise Experimental

4.1 Performance — Colisões

Documento	Hash ASCII	Hash Polinomial
Doc1	80	76
Doc2	74	74
Doc3	61	73
Doc4	78	79
Doc5	107	110

Como explicado anteriormente, a quantidade de colisões depende muito do tamanho do vocabulário.

4.2 Performance — Métricas de Similaridade

Pares	Cosseno	Jaccard	Dice
Doc1 - Doc2	0,46	0,25	0,4
Doc1 - Doc3	0,16	0,05	0,1
Doc1 - Doc4	0,12	0,03	0,06
Doc1 - Doc 5	0,19	0,02	0,04
Doc2 - Doc3	0,16	0,06	0,11
Doc2 - Doc4	0,2	0,06	0,11
Doc2 - Doc5	0	0	0
Doc3 - Doc4	0,52	0,27	0,42
Doc3 - Doc5	0,05	0,01	0,02
Doc4 - Doc5	0,01	0,01	0,02

Ambos trazem maior e menor número de similaridade para os mesmos documentos, mostrando efetividade nesse ponto nas 3 métricas. .

4.3 Performance — Rotações

Pares	Rotações Simples	Rotações Duplas
Doc1 - Doc2	0	0
Doc1 - Doc3	0	0
Doc1 - Doc4	1	0
Doc1 - Doc 5	0	0
Doc2 - Doc3	0	0
Doc2 - Doc4	1	0
Doc2 - Doc5	1	0
Doc3 - Doc4	0	0
Doc3 - Doc5	0	0
Doc4 - Doc5	0	1
Total	3	1

- . O pequeno número de rotações indica que os valores de similaridade entre os documentos testados apresentaram boa distribuição, não são tão similares..

5. Resultados e Discussões

5.1 Pares mais similares nos testes:

Doc3 $\Leftrightarrow$ Doc4 = 0,52 de similaridade com Cosseno

Doc1 $\Leftrightarrow$ Doc2 = 0,46 de similaridade com Cosseno

5.2 “Qual seria o impacto (em termos de complexidade e adequação ao problema) de substituir a Árvore AVL por outra estrutura, como uma heap?”

Operação	AVL	Heap
Inserção	$O(\log n)$	$O(\log n)$
Busca por valor	$O(\log n)$	$O(n)$
Maior/Menor elemento	$O(\log n)$	$O(1)$

Analisando a tabela acima, podemos ver que uma estrutura heap poderia sim ser útil para o projeto, principalmente na função de TopK, que teria complexidade de $O(1)$. Porém, no modo lista nós precisamos que os pares sejam ordenados acima de uma limiar, a heap deixaria a desejar visto que ela não mantém a ordenação completa, só garante que o topo é o maior. O modo busca também se mostra mais eficiente com a AVL, visto que precisamos buscar um valor específico. Além de que a travessia da AVL já nos retorna os valores ordenados naturalmente.

Ou seja, de modo geral, a AVL se mostra eficiente para todas as operações que precisamos no projeto, enquanto a Heap seria muito útil apenas para a TopK.

5.3 “Como as etapas de pré-processamento de texto (normalização, remoção de stop words, etc.) ajudaram a evitar falsos positivos de similaridade? Que outros passos poderiam ser tomados?”

-Conversão para minúsculas: Palavras iguais com diferença de letras minúsculas ou maiúsculas poderiam inflar o vocabulário, afetando a verdadeira similaridade entre os documentos;

-Remoção de Pontuação: Mesmo raciocínio da conversão para minúsculas, palavras do tipo “ola,” seriam consideradas diferentes da palavra “ola”;

-Stop Words: Isso, ao contrário dos pontos anteriores, inflaria a própria similaridade. Documentos

de assuntos distintos poderiam ter uma grande similaridade apenas por terem muitas palavras “de”, “ou”, “e”, etc;

-Tokenização: garante que o texto seja dividido em unidades significativas comparáveis.

Outros passos a serem tomados:

-Stemming: Poderia reduzir palavras a sua “raiz”, possibilitando detectar similaridade entre palavras do tipo “computação”, “computador”, “computacional” e etc;

-Remoção de Números: Números raramente trazem algum significado capaz de definir a similaridade entre documentos.

6. Conclusão

A implementação cumpriu o que o enunciado da atividade propôs. O programa é capaz de comparar documentos com duas funções hashes distintas e com 3 tipos de métricas de similaridade diferentes. Encontramos duas funções de hash que trazem um número baixo de colisões levando em consideração o tamanho dos documentos utilizados, mesmo tendo o obstáculo de ter que trabalhar com uma chave do tipo String, obrigando a utilizar os códigos ASCII dos caracteres para calcular a função de Hash. Além disso, também encontramos uma certa dificuldade na implementação de listas dentro dos nós para que a AVL não ignorasse as duplicatas, e sim, adicionasse o par à lista do nó correspondente à frequência. Tivemos um pequeno “bug” na hora de fazer a contagem de colisões, o programa estava retornando um número muito alto. Após analisar a classe HashTable, descobrimos que a função que colocamos o contador de colisões era chamada por outras 3 funções (put, get e remove). Foi então que pensamos na ideia de adicionar um argumento booleano nessa função (procurarPosicao), então quando put a chamava, passava true como argumento, enquanto as outras duas funções passavam false, permitindo que o incremento fosse realizado somente quando um elemento fosse inserido.
